

Chapter 10

Dynamic Networks

10.1 Introduction

Most complex systems—and, hence, networks—are dynamic in nature. So, realistically, the corresponding network graphs and processes thereon are dynamic as well and, ideally, should be analyzed as such. Friendships (both traditional and on-line versions) form and dissolve over time. Certain genes may regulate other genes, but only during specific stages of the natural cycle of a cell. And both the physical and logical structure of the Internet have been evolving ever since it was first constructed.

In practice, however, the vast majority of network analyses performed to date have been static. Arguably this state of affairs is due to a number of factors. First of all, broadly speaking, the methodology for dynamic network analysis is decidedly less developed than that for static network analysis. At the same time, unless handled carefully (and, sometimes, even then!), the computational burden associated with the analysis of dynamic network data can be comparatively much heavier. Additionally, it is not necessarily always the case that a complex system is varying in ways interesting or relevant to a study at the time scales upon which dynamics can be measured. That is, oftentimes a handful of well-performed static network analyses can sufficiently convey the behavior of the underlying system across time. And finally, sometimes it simply is not realistic to collect data indexed over time. For example, in systems biology it generally is far easier to collect measurements on gene transcription activity in a single, steady-state setting than at consecutive time points, particularly if temporal calibration across multiple cells is required.

Accordingly, the focus of this book has been devoted almost entirely to statistical methods for static network data. Nevertheless, given the importance of dynamic network data, and the expectation that the development of computationally efficient methods for its analysis will continue to receive increasingly greater attention in the coming years, we present here in this chapter a brief introduction to this quickly evolving area. The organization of the sections in this chapter mimics that

of the earlier chapters in this book. Specifically, we discuss the representation and manipulation of dynamic network data in Sect. 10.2, its visualization in Sect. 10.3, its characterization in Sect. 10.4, and finally its modeling in Sect. 10.5.

10.2 Representation and Manipulation of Dynamic Networks

The adjective ‘dynamic’ has been applied in the context of networks to describe at least two different aspects of an evolving complex system. Most commonly, it is applied when the edges among a set of vertices—and sometimes the set of vertices itself—are changing as a function of time. Alternatively, it is sometimes used in reference to the attributes of the vertices or edges in a fixed graph G changing in time, such as in the case of the dynamic network processes discussed in Chap. 8.5 (e.g., epidemics). The former case may be thought of as referring to dynamics *of* a network, and the latter, to dynamics *on* a network. Of course, both types of dynamics may be present together—networks of this type sometimes are referred to as *co-evolving*. In this chapter our focus will be primarily on the first of the above cases.

Conceptually, we will think of a *dynamic network* as a time-indexed graph $G(t) = (V(t), E(t))$, with time t varying in either a discrete or continuous manner over some range of values. Here $V(t)$ is the set of vertices present in the network at time t , and $E(t)$, the set values $e_{ij}(t)$, for $i, j \in V(t)$, indicating the presence or absence of edges between those vertices. Sometimes it is convenient to let V be the collection of all vertices present at some point or another during the period of time under study, in which case we can write $V(t) \equiv V$ for all t .

In practice, the extent to which the time-varying behavior of a dynamic network $G(t)$ actually may be captured through measurements can vary. At one extreme, we may be able to observe with complete accuracy the appearance and disappearance of each vertex in time, and similarly the formation and dissolution of edges between the vertices. That is, we may literally observe $G(t)$. And, of course, at the other extreme, we may only be able to obtain a marginal summary of those vertices and edges that appeared at any point during the period of observation, resulting in a static graph G . In between these two extremes lie various other possibilities. For example, we may observe a set of static snapshots of the network, each summarizing the marginal behavior of the network during successive time periods. Such data are sometimes referred to as panel data or longitudinal network data. Alternatively, we may have knowledge of interactions between vertices only up to some finite resolution in time.

An example of this last type of data is found in the data set `hc`.

```
#10.1 1 > library(sand)
      2 > data(hc)
```

These data,¹ first reported by Vanhems et al. [141], contain records of contacts among patients and various types of health care workers in the geriatric unit of a hospital in Lyon, France, in 2010, from 1pm on Monday, December 6 to 2pm on

¹ Original data are available at <http://www.sociopatterns.org>.

Friday, December 10. Each of the 75 people in this study consented to wear RFID sensors on small identification badges during this period, which made it possible to record when any two of them were in face-to-face contact with each other (i.e., within 1–1.5 m of each other) during a 20-second interval of time. A primary goal of this study was to gain insight into the pattern of contacts in such a hospital environment, particularly with an eye towards the manner in which infection might be transmitted.

The object `hc` is a data frame in which the measurements are stored as an edge list.

```
#10.2 1 > head(hc)
      2   Time ID1 ID2  S1  S2
      3 1   140  15  31 MED ADM
      4 2   160  15  22 MED MED
      5 3   500  15  16 MED MED
      6 4   520  15  16 MED MED
      7 5   560  16  22 MED MED
      8 6   580  16  22 MED MED
```

For each contact during the week-long period of observation there is a row, in which are recorded the time (in seconds) at which the corresponding 20-second interval terminated, an identification number for each of the two people involved in the contact, and the statuses of these people. Status was assigned according to one of four categories, i.e., administrative staff (ADM), medical doctor (MED), paramedical staff, such as nurses or nurses' aides (NUR), and patients (PAT). We note that the designation of '1' and '2' in the ID and status variables is arbitrary.

From a network-based perspective, we view the people in this study as vertices, and their status, as a vertex attribute. In order to get a sense of the distribution of this attribute across vertices, we merge the vertex IDs and status variables, extract the unique pairings of ID and status, and summarize the results in a table.

```
#10.3 1 > ID.stack <- c(hc$ID1, hc$ID2)
      2 > Status.stack <- c(as.character(hc$S1),
      3 +   as.character(hc$S2))
      4 > my.t <- table(ID.stack, Status.stack)
      5 > v.status <- character(nrow(my.t))
      6 > for(i in (1:length(v.status))) {
      7 +   v.status[i] <- names(which(my.t[i,] != 0))
      8 + }
      9 > table(v.status)
     10 v.status
     11 ADM MED NUR PAT
     12 8  11 27 29
```

We see that patients and nurses each make up roughly 40% of the study, while the administrators and medical doctors each make up roughly only 10%.

Similarly, we can view each contact between two people as an edge. Summarizing these contacts in a two-way table allows us to get some rough preliminary sense as to the distribution of such edges throughout our network.

```
#10.4 1 > status.t <- table(hc$S1, hc$S2)
2 > status.t <- status.t + t(status.t)
3 > diag(status.t) <- round(diag(status.t)/2)
4 > status.t
5           ADM    MED    NUR    PAT
6  ADM    279    459   2596    441
7  MED    459   5660   1769   1471
8  NUR   2596   1769  12695   6845
9  PAT    441   1471   6845    209
```

We see, for example, that easily the largest number of contacts was between nurses.

A more refined sense of the pattern of contacts among these individuals may be obtained by incorporating time into our analysis. Figure 10.1 shows histograms of these contacts, as a function of time, grouped by the status of the interacting pairs.

```
#10.5 1 > tmp.es <- paste(hc$S1, "-", hc$S2, sep="")
2 > e.status <- character(dim(hc)[1])
3 > e.status[tmp.es=="ADM-ADM"] <- "ADM-ADM"
4 > e.status[tmp.es=="MED-MED"] <- "MED-MED"
5 > e.status[tmp.es=="NUR-NUR"] <- "NUR-NUR"
6 > e.status[tmp.es=="PAT-PAT"] <- "PAT-PAT"
7 > e.status[(tmp.es=="ADM-MED") |
8 + (tmp.es=="MED-ADM")] <- "ADM-MED"
9 > e.status[(tmp.es=="ADM-NUR") |
10 + (tmp.es=="NUR-ADM")] <- "ADM-NUR"
11 > e.status[(tmp.es=="ADM-PAT") |
12 + (tmp.es=="PAT-ADM")] <- "ADM-PAT"
13 > e.status[(tmp.es=="MED-NUR") |
14 + (tmp.es=="NUR-MED")] <- "MED-NUR"
15 > e.status[(tmp.es=="MED-PAT") |
16 + (tmp.es=="PAT-MED")] <- "MED-PAT"
17 > e.status[(tmp.es=="NUR-PAT") |
18 + (tmp.es=="PAT-NUR")] <- "NUR-PAT"
19 >
20 > my.hc <- data.frame(Time = hc$Time/(60*60),
21 +                     ID1 = hc$ID1,
22 +                     ID2 = hc$ID2,
23 +                     Status = e.status)
24 >
25 > library(lattice)
26 > histogram( ~ Time|Status, data=my.hc, xlab="Hours",
27 +           layout=c(5,2))
```

Several interesting features now become apparent. For example, there is a clear diurnal structure to most of the histograms, with the vast majority of the contacts—not surprisingly—having occurred during the daytime or early evening hours. There are also exceptions to this pattern, the most notable of which may be the concentration of contacts between pairs of administrators (i.e., ADM–ADM) on the last day, suggesting the likelihood of a weekly organizational meeting.

Note that so far the notion of a network has only been implicit in our discussion. There are in fact several ways in which such data may be provided with an explicit

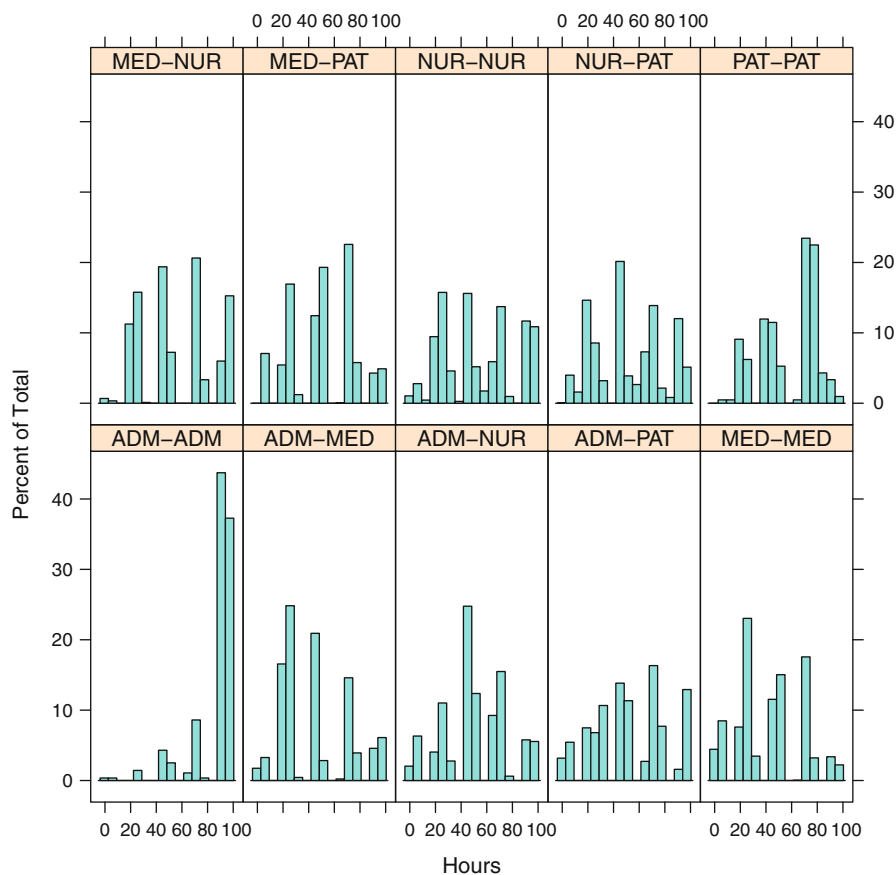


Fig. 10.1 Histograms of contacts in the hospital data set *hc*, grouped by the status of the interacting pair

network-based representation. One straightforward approach is through the use of multi-graphs, where each contact is represented by an edge, and each edge is equipped with an attribute indicating the time interval associated with it. This representation can be implemented in a straightforward fashion in **igraph**. Applying it to our hospital data

```
#10.6 1 > vids <- sort(unique(c(hc$ID1, hc$ID2)))
2 > g.week <- graph.data.frame(hc[, c("ID1", "ID2",
3 +   "Time")], vertices=data.frame(vids),
4 +   directed=FALSE)
5 > E(g.week)$Time <- E(g.week)$Time / (60 * 60)
```

the resulting network is a single graph of 32,424 (multi)edges, annotated with time information, among 75 vertices.

```
#10.7 1 > g.week
      2 IGRAPH UN-- 75 32424 --
      3 + attr: name (v/c), Time (e/n)
```

We also incorporate status as a vertex attribute.

```
#10.8 1 > status <- unique(rbind(data.frame(id=hc$ID1,
      2 +   status=hc$S1), data.frame(id=hc$ID2, status=hc$S2)))
      3 > V(g.week)$Status <-
      4 +   as.character(status[order(status[,1]),2])
```

Alternatively, we can represent such data in the form of a simple graph, by collapsing the multiple edges between vertex pairs to a single edge, but preserving information on the multiplicity of edges through the use of edge weights. We do so for the hospital data.

```
#10.9 1 > E(g.week)$weight <- 1
      2 > g.week.wgtd <- simplify(g.week)
      3 > g.week.wgtd
      4 IGRAPH UNW- 75 1139 --
      5 + attr: name (v/c), Status (v/c), weight (e/n)
```

We see that the resulting graph is indeed simple

```
#10.10 1 > is.simple(g.week.wgtd)
      2 [1] TRUE
```

and that its 1,139 edges have weights varying over a number of orders of magnitude, although the vast majority are on the order of 1–20.

```
#10.11 1 > summary(E(g.week.wgtd)$weight)
      2   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      3   1.00   3.00   8.00  28.47  23.00 1059.00
```

Note that the time information is completely lost in collapsing a multi-graph to a weighted graph. A compromise is to create ‘slices’ of the multi-graph, and collapse within each slice. For example, we might take the first four days (96 h) of our hospital data

```
#10.12 1 > g.week.96 <- subgraph.edges(g.week,
      2 +   E(g.week)[Time <= 96])
```

and separate that into eight slices of 12 h each.

```
#10.13 1 > g.sl12 <- lapply(1:8, function(i) {
      2 +   g <- subgraph.edges(g.week,
      3 +   E(g.week)[Time > 12*(i-1) &
      4 +   Time <= 12*i],
      5 +   delete.vertices=FALSE)
      6 +   simplify(g)
      7 + })
```

The resulting eight subnetworks are stored as a list of graphs. Each of these graphs has been defined to include the full set of 75 people as its set of vertices²

```
#10.14 1 > sapply(g.sl12,vcount)
      2 [1] 75 75 75 75 75 75 75 75
```

and is seen to contain on the order of two to three hundred pairs of vertices for whom the corresponding people were involved in at least one contact.

```
#10.15 1 > sapply(g.sl12,ecount)
      2 [1] 179 294 257 282 265 314 197 305
```

Finally, for measurements with sufficiently high temporal resolution—that is, for when we effectively observe $G(t) = (V(t), E(t))$ itself—it is natural to wish to maintain this representation of the dynamic network. Recently released as of this writing, the package **networkDynamic** allows for such a representation.

```
#10.16 1 > library(networkDynamic)
```

Specifically, each period during which a dynamic edge $e_{ij}(t)$ is ‘on’—what the authors of this package refer to as a ‘spell’—is represented by its onset and termination. The dynamic network as a whole is then represented as a collection of such spells.

To illustrate, with our hospital data `hc`, for each contact we might extend the contact period time index back by 20 s, indicating a tacit assumption that the corresponding contact lasted the entire 20-second period (in reality we know only that a contact occurred sometime during that period). Then for each pair of vertices in our network, a spell would be defined by the union of consecutive time periods, with the minimum length of any spell being 20 s, by definition.

```
#10.17 1 > hc.spls <- cbind((hc$Time-20)/(60*60),
      2 + hc$Time/(60*60),
      3 + hc$ID1, hc$ID2)
      4 > hc.dn <- networkDynamic(edge.spells=hc.spls)
```

Within **networkDynamic** it is then possible to query the network in various ways. For example, we can discover that the first edge (defined as the edge in our data with the earliest spell), while active during the first hour, was not active during the second hour.

```
#10.18 1 > is.active(hc.dn, onset=0, terminus=1, e=c(1))
      2 [1] TRUE
      3 > is.active(hc.dn, onset=1, terminus=2, e=c(1))
      4 [1] FALSE
```

Similarly, for any given edge we can extract the full set of spells during which it was active. For instance, the first edge was active only once in fact, at the very start of our 4-day period.

² Using the option `delete.vertices=TRUE` in the function `subgraph.edges` would instead result in graphs with vertex subsets corresponding only to those people involved in at least one contact during a given 12-hour period.

```
#10.19 1 > get.edge.activity(hc.dn, e=c(1))
2 [[1]]
3           [,1]           [,2]
4 [1,] 0.03333333 0.03888889
```

On the other hand, the tenth edge was involved in seven contacts (i.e., seven spells of the minimal length of 20s), all of which were confined to a roughly 24 h period.

```
#10.20 1 > get.edge.activity(hc.dn, e=c(10))
2 [[1]]
3           [,1]           [,2]
4 [1,] 0.800000 0.8055556
5 [2,] 1.355556 1.3611111
6 [3,] 1.505556 1.5111111
7 [4,] 24.894444 24.905556
8 [5,] 25.005556 25.0166667
9 [6,] 25.388889 25.3944444
10 [7,] 25.500000 25.5055556
```

It is also possible within **networkDynamic** to convert between various different network-based representations. For example, analogous to earlier in this section, we can produce eight subnetworks corresponding to successive 12-hour periods.

```
#10.21 1 > g.sl12.dN <- get.networks(hc.dn, start=0, end=96,
2 +                               time.increment=12)
```

Alternatively, we can summarize the network in the form of a data frame.

```
#10.22 1 > hc.dn.df <- as.data.frame(hc.dn)
2 > names(hc.dn.df)
3 [1] "onset" "terminus"
4 [3] "tail" "head"
5 [5] "onset.censored" "terminus.censored"
6 [7] "duration" "edge.id"
```

Here the rows correspond to spells, rather than contacts. We find that the majority of spells in fact lasted no more than one or at most two consecutive contact periods of 20s. In contrast, the longest lasted for a bit over an hour.

```
#10.23 1 > summary(hc.dn.df$duration)
2      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
3 0.005556 0.005556 0.005556 0.012830 0.011110 1.089000
```

10.3 Visualization of Dynamic Networks

With the various possible dynamic network representations just discussed there come a similar variety of ways in which dynamic networks may be visualized. Obviously the vast majority of the basic principles of graph drawing or visualization that were discussed in Chap. 3, for static network graphs, continue to hold true in the dynamic setting. However, some additional concerns arise as well in the latter case, around the question of how best to incorporate the notion of time.

Of course, we can ignore time altogether and simply visualize either a multi-graph representation or the corresponding weighted graph obtained by collapsing multi-edges. For the hospital data, plotting a multi-graph (not shown) with on the order of 32 thousand edges among just 75 vertices yields a visualization that is essentially useless. A visualization of the corresponding weighted graph, shown in Fig. 10.2, is somewhat better, but is still much too busy for us to gain any real insight.

```
#10.24 1 > detach(package:networkDynamic)
2 > l = layout.fruchterman.reingold(g.week.wgtd)
3 > v.cols <- character(75)
4 > v.cols[V(g.week.wgtd)$Status=="ADM"] <- "yellow"
5 > v.cols[V(g.week.wgtd)$Status=="MED"] <- "blue"
6 > v.cols[V(g.week.wgtd)$Status=="NUR"] <- "green"
7 > v.cols[V(g.week.wgtd)$Status=="PAT"] <- "black"
8 > plot(g.week.wgtd, layout=l, vertex.size=3,
9 +     edge.width=2*(E(g.week.wgtd)$weight)/100,
10 +     vertex.color=v.cols, vertex.label=NA,)
```

Ideally, we would like to maintain as much of the available temporal information as possible when visualizing dynamic network data. From the point of view of the graph display problem, a fundamental challenge in moving from the static to the dynamic is the need to respect, in the case of the latter, what is referred to as the user's *mental map*. This term is used to describe the result of the process by which, upon studying a given static network map, a user becomes familiar with it, interprets it, and navigates about it. Simply put, we would expect a certain amount of 'stability' across visualizations.

When the dynamic network is represented as a sequence of 'slices', whether due to measurement constraints or user choice, and the number of slices is not too large, it may be useful to look at the corresponding sequence of static graph visualizations. In this case, it is natural to encourage a sense of stability by maintaining the positions of vertices across visualizations.

For example, in Fig. 10.3 we show plots of the hospital contact network as a sequence of eight static plots, each corresponding to a 12-hour time period.

```
#10.25 1 > opar <- par()
2 > par(mfrow=c(2,4),
3 +     mar=c(0.5, 0.5, 0.5, 0.5),
4 +     oma=c(0.5, 1.0, 0.5, 0))
5 > for(i in (1:8)) {
6 +   plot(g.s112[[i]], layout=l, vertex.size=5,
7 +       edge.width=2*(E(g.week.wgtd)$weight)/1000,
8 +       vertex.color=v.cols, vertex.label=NA)
9 +   title(paste(12*(i-1), "to", 12*i, "hrs"))
10 + }
11 > par(opar)
```

More ambitiously, if the data are rich enough temporally, instead of individually displaying each of the many corresponding individual static graphs, one might animate them to produce a 'movie,' so as to watch the network graphs change over time. Although the area of dynamic graph drawing of this type is still arguably in its

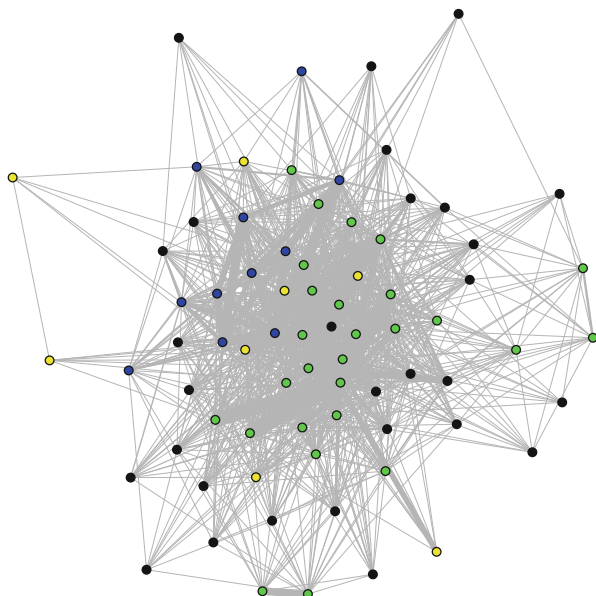


Fig. 10.2 Visualization of the hospital contact data, based on a weighted graph summarizing all contacts over the full week. Vertex colors indicate status of each person: ADM (yellow), MED (blue), NUR (green), and PAT (black)

infancy, there are already a number of approaches that have been developed for automated display of dynamic network graphs with constraints to reinforce a consistent user mental map. Broadly speaking, rather than stringently refusing to allow vertices and edges to move, as above, one attempts to allow only ‘local’ elements in the vicinity of a newly added graph element to change.

Within R, at the time of this writing the only package that allows for the production of such dynamic network visualizations is **ndtv**, which takes as its input an object from **networkDynamic**. However, it should be noted that the process of creating a dynamic layout can take considerably more time than in the static case, and the authors of **ndtv** caution that its use currently is most realistic for networks of relatively small size. For example, it is unrealistic to apply to our hospital contact data.

On the other hand, it is possible to produce a useful visualization of primarily the temporal information in dynamic network data in a relatively straightforward manner, if connectivity beyond vertex pairs (i.e., edges) is suppressed. Specifically, we can display the onset and termination of edges as a function of time, in the form of a type of phase-plot, with the edges indexed along the y-axis in order of their first appearance in the network. A visualization of this type is shown in Fig. 10.4, for the hospital contact data.³

³ The function `timeline` in **ndtv** can be used to produce a similar plot, but currently does not have the functionality to permit annotation by edge attributes, as we require here.

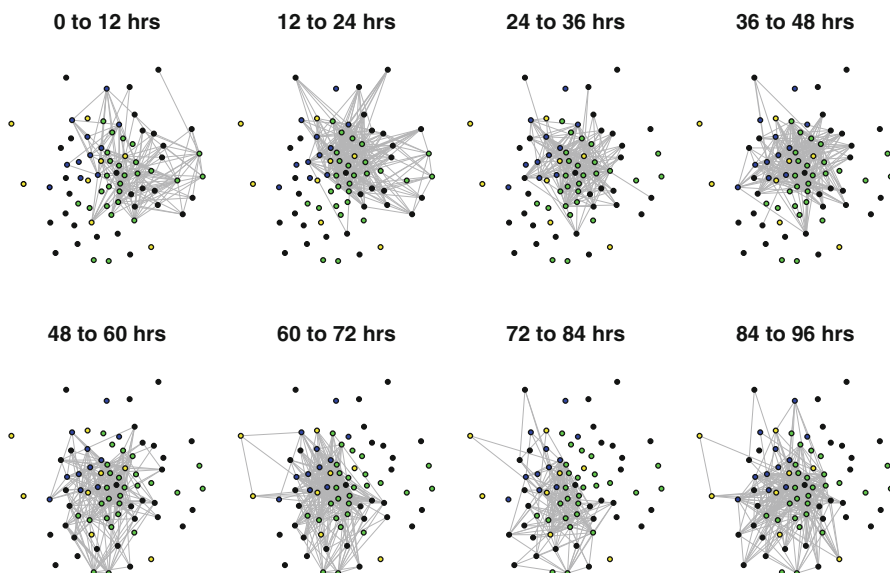


Fig. 10.3 Visualization of the hospital contact data, as a sequence of eight static networks corresponding to consecutive 12-hour periods. Vertex colors indicate status of each person: ADM (yellow), MED (blue), NUR (green), and PAT (black)

```
#10.26 1 > # Establish colors for edge status.
2 > tmp.es <- paste(v.status[hc.dn.df$tail], "-",
3 +               v.status[hc.dn.df$head], sep="")
4 > mycols <- numeric(nrow(hc.dn.df))
5 > mycols[tmp.es=="ADM-ADM"] <- 1
6 > mycols[tmp.es=="MED-MED"] <- 2
7 > mycols[tmp.es=="NUR-NUR"] <- 3
8 > mycols[tmp.es=="PAT-PAT"] <- 4
9 > mycols[(tmp.es=="ADM-MED") | (tmp.es=="MED-ADM")] <- 5
10 > mycols[(tmp.es=="ADM-NUR") | (tmp.es=="NUR-ADM")] <- 6
11 > mycols[(tmp.es=="ADM-PAT") | (tmp.es=="PAT-ADM")] <- 7
12 > mycols[(tmp.es=="MED-NUR") | (tmp.es=="NUR-MED")] <- 8
13 > mycols[(tmp.es=="MED-PAT") | (tmp.es=="PAT-MED")] <- 9
14 > mycols[(tmp.es=="NUR-PAT") | (tmp.es=="PAT-NUR")] <- 10
15 > my.palette <- rainbow(10)
16 > # Produce plot.
17 > ne <- max(hc.dn.df$edge.id)
18 > max.t <- max(hc.dn.df$terminus)
19 > plot(c(0, max.t), c(0, ne), ann=F, type='n')
20 > segments(hc.dn.df$onset, hc.dn.df$edge.id,
21 +         hc.dn.df$terminus, hc.dn.df$edge.id,
22 +         col=my.palette[mycols])
23 > title(xlab="Time (hours)",
24 +       ylab="Interacting Pair
25 +       (Ordered by First Interaction)")
```

```

26 > abline(v=c(11, 35, 59, 83), lty="dashed", lw=2,
27 +       col="lightgray")
28 > # Add legend to plot.
29 > status.pairs <- c("ADM-ADM", "MED-MED", "NUR-NUR",
30 + "PAT-PAT", "ADM-MED", "ADM-NUR", "ADM-PAT",
31 + "MED-NUR", "MED-PAT", "NUR-PAT")
32 > legend(7,1170,status.pairs,
33 +       text.col=my.palette[(1:10)],cex=0.75)

```

Some of the temporal information in Fig. 10.1 is evident as well here in Fig. 10.4. For example, the diurnal patterns in contacts are clear. However, here we now have the capability to track individual edges and, through the use of color, edges of various combinations of statuses. For example, we see that there is a cluster of contacts between nurses that are introduced in the second half of the 2 day (i.e., edges numbered roughly 500, just after hour twenty) that are not seen again until late in the last day.

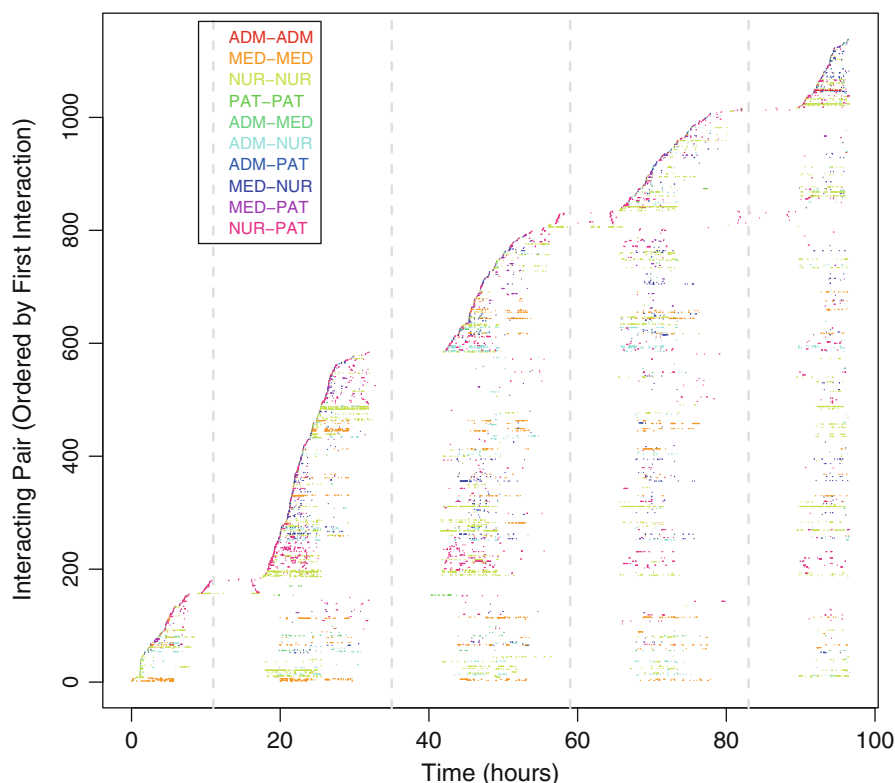


Fig. 10.4 Timeline showing the presence/absence of dynamic edges in the hospital contact network, covering just over 96 h over 5 days (midnight of each day indicated by *gray vertical lines*)

10.4 Characterization of Dynamic Networks

The development of a comprehensive body of tools for characterizing dynamic networks, and of a corresponding understanding of their properties and usage, currently lags far behind that for static graphs G , such as was described in Chap. 4. This lacking is probably due in part to insufficiently pressing need, in that traditionally it has been notably more difficult in most network studies to obtain time-indexed network data that are rich in both quality and quantity. As we have noted, however, this situation is beginning to change, and the demand for tools to characterize dynamic networks is growing. Also contributing to the gap between static and dynamic methods of characterization is possibly the absence of a similarly mature graph-theoretic infrastructure. Finally, another important factor is the sheer combinatorial explosion in problem variants, once the factor of ‘time’ is to be incorporated. See Moody, McFarland, and Bender-deMoll [114], for example, for a detailed discussion of how and why such variants arise, in the context of social networks.

At present, therefore, arguably the most common manner in which dynamic networks are characterized in practice is through the application of methods for static networks to successive slices over time. Just as in the static case, the choice of what methods to use is specific to the application at hand and should be driven by particular questions about the complex system being studied. Here we illustrate this approach to dynamic network characterization using the hospital contact data again, where recall that the goal was to obtain insight into the manner in which infection might be transmitted between individuals in this environment.

In Chap. 8.5.1 we saw that network topology can affect the spread of a disease. And we know from Chap. 4 that the degree distribution is one of the most fundamental summaries of network topology. Thus, to begin the process of understanding the potential spread of infection in the hospital from which these data were obtained, we examine the degree distributions for each of the eight successive 12-hour periods.

```
#10.27 1 > all.deg <- sapply(g.sl12, degree)
      2 > sl.lab<- unlist(lapply(1:8, function(i)
      3 +   paste(12*(i-1), "-", 12*i, "hrs", sep="")))
      4 > deg.df <- data.frame(Degree=as.vector(all.deg),
      5 +   Slice = rep(sl.lab, each=75),
      6 +   Status = rep(V(g.week)$Status, times=8))
      7 > library(ggplot2)
      8 > p = qplot(factor(Degree), data=deg.df,
      9 +   geom="bar", fill=Status)
     10 > p + facet_grid(Slice ~ .)+xlab("Degree")+ylab("Count")
```

In Fig. 10.5 are shown the resulting displays of the degree distributions, visualized as bar charts, where the bars have been colored to indicate the relative frequency of vertices of each of the four statuses. Several interesting features are evident. We see, for example, a large mode at zero in each bar chart. During any 12-hour period there is a substantial fraction of the 75 people studied that were not involved in any recorded contacts. Among hospital employees (i.e., ADM, MED, and NUR) this pattern presumably reflects duty shifts. More interestingly, we see that the vast

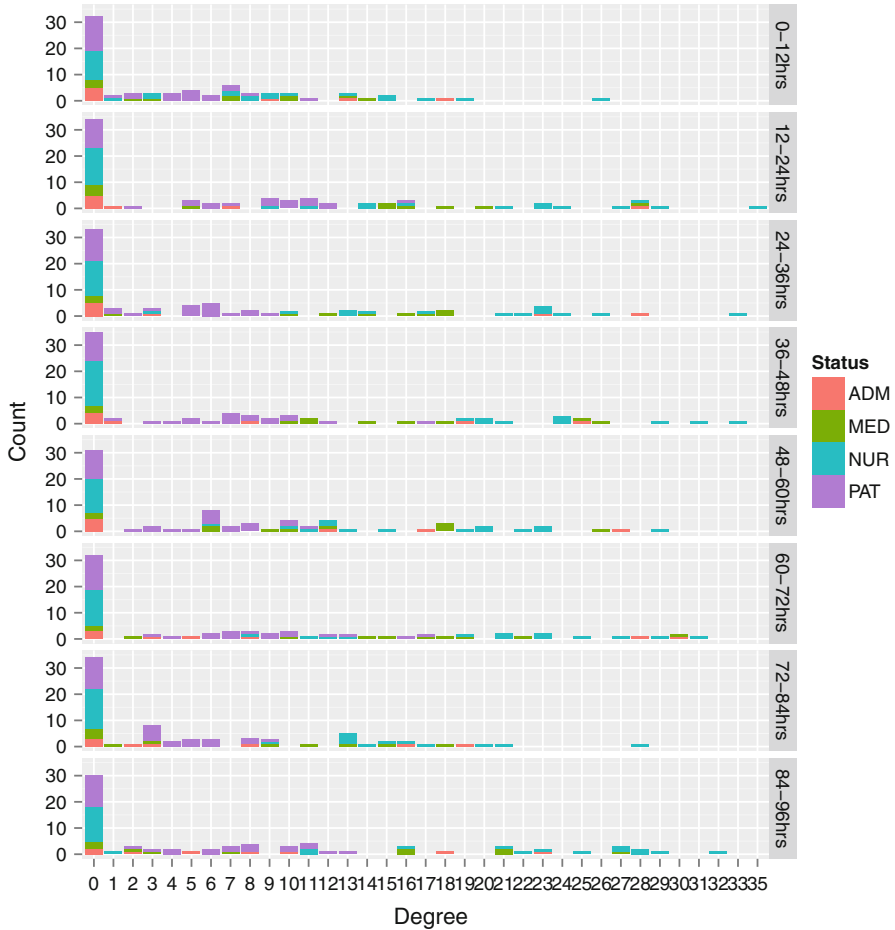


Fig. 10.5 Degree distributions for the hospital contact data, one per each of eight static networks corresponding to consecutive 12-hour periods

majority of patients (i.e., PAT) are involved in contacts with comparatively fewer numbers of other people. Similarly, the hospital employees tend to have contacts with higher numbers of people.

This exploration suggests that the most pervasive potential spreaders of disease would be among the hospital employees. In order to try to identify such individuals, and acknowledging that they may change from day to day, we create a summary of who is consistently among the top five (i.e., has the fifth highest degree or larger) over the week.

```
#10.28 1 > top.deg <- lapply(1:8,function(i) {
2 +   all.deg[,i][rank(all.deg[,i])>=70]
3 + })
4 > table(unlist(lapply(1:8,function(i)
5 +   as.numeric(names(top.deg[[i]]))))))
```

```

6  1  5  7  8 10 11 13 15 17 19 21 22 23 24 25 26 27 29 31
7  2  2  4  1  1  2  2  4  3  3  2  1  3  2  1  2  2  2  1
8 34 36 37 63 64
9  1  1  2  1  2

```

We see that two of the people in this network (i.e., with vertex IDs 7 and 15) are among the top five during four of the eight successive 12-hour periods. And these two are indeed among the hospital employees, consisting of a nurse and a doctor, respectively.

```

#10.29 1 > V(g.week)$Status[c(7,15)]
2 [1] "NUR" "MED"

```

Next, for these two individuals, we look at the relative number of contacts per person contacted (i.e., the number of contacts per alter, for each of these two egos). This means looking at ratio of vertex strength to degree.

```

#10.30 1 > all.str <- sapply(g.sl12, graph.strength)
2 > all.r <- all.str/all.deg
3 > round(all.r[c(7, 15),], digits=2)
4      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
5 7      2.00 25.79 11.1 32.73 14.20 33.19  8.33 37.34
6 15     29.71 26.33 17.0 12.48 19.27 15.30 19.40 12.93

```

We see that, compared to the network as a whole,

```

#10.31 1 > summary(c(all.r))
2      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
3      1.00   5.00   10.43   12.06  15.73   47.35    261

```

this nurse and this doctor tend largely to have a rate of contacts at or above the mean, and more often than not, in the upper quartile. So not only do these two individuals come into contact with many more others than usual in this hospital ward, they also tend to have noticeably higher rates of contact with each, on average.

Accordingly, one might argue that these two individuals are natural candidates to consider as potential ‘super-spreaders’. It would then be natural to also consider just how quickly an infection might spread from these two. One rough indicator is the average shortest-path distance from each of these two to the rest of the individuals in the network. Again calculating this quantity slice-by-slice,

```

#10.32 1 > sp.len <- lapply(1:8, function(i) {
2 +   spl <- shortest.paths(g.sl12[[i]], v=c(7, 15),
3 +                               to=V(g.sl12[[i]]),
4 +                               weights=NA)
5 +   spl[spl==Inf] <- NA
6 +   spl
7 + })
8 > ave.spl <- sapply(1:8, function(i)
9 +   apply(sp.len[[i]], 1, mean, na.rm=T))
10 > round(ave.spl, digits=2)
11      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
12 7      3.05 1.27 1.79 1.12 1.80 1.33 2.00 1.24
13 15     1.72 1.51 2.48 1.35 1.36 1.26 1.61 1.36

```

we find that for both individuals in most slices their distance on average from the rest of the network—among those having at least one contact during that slice—is less than two. These numbers might seem quite small compared to the overall diameter of the networks.

```
#10.33 1 > sapply(g.sl12, diameter)
      2 [1]  9  8 26 28 10 10 10 10
```

However, compared to the overall average shortest path distance, the contrast is decidedly less stark.

```
#10.34 1 > round(sapply(g.sl12, average.path.length), digits=2)
      2 [1] 2.12 1.70 1.81 1.67 1.79 1.70 1.91 1.78
```

Nevertheless, both the nurse and the doctor do still seem to be somewhat closer to the others in the network most of the time, which could be argued to likely have a further detrimental effect on containing infection, were either of them to become infected themselves.

10.5 Modeling Dynamic Networks

The modeling of dynamic network data is nontrivial. The complexity that we have already seen associated with modeling a static network graph, in Chaps. 5 and 6, is further magnified by the need to not only model ‘many’ such graphs (i.e., in a discrete or a continuous sense, as the case may be), but also—in particular—by the need to model at the same time the manner in which these graphs evolve from one to another. Compared to the existing literature on modeling of static network graphs, there is decidedly less development on the dynamic side. But this state of affairs is quickly changing. However, as of this writing, there is little available in $\mathbf{R}$. And, in addition, a sufficiently careful treatment of the few dynamic network modeling frameworks that are available in $\mathbf{R}$ would require substantial attention and is beyond the scope of this book. Therefore, we will restrict ourselves here simply to summarizing two available options.

Given the important role that exponential random graph models (ERGMs) have played in the modeling of static network graphs, as described in Chap. 6, they have been considered a natural candidate for extension to the case of dynamic networks. In recent work, building upon and modifying earlier work of Hanneke and Xing [69] and Hanneke et al. [70], Krivitsky and Handcock [94] have introduced a class of temporal exponential random graph models for longitudinal network data (i.e., in which networks are observed in panels, like the ‘slices’ we created for the hospital data), in which the formation and dissolution of edges are modeled in a separable fashion.

Specifically, let $\mathbf{Y}(t)$ and $\mathbf{Y}(t-1)$ denote the adjacency matrices for the network at times t and $t-1$, and (in a slight abuse of notation) define the so-called formation network $\mathbf{Y}^+ = \mathbf{Y}(t) \cup \mathbf{Y}(t-1)$ and dissolution network $\mathbf{Y}^- = \mathbf{Y}(t) \cap \mathbf{Y}(t-1)$ relative to time point t . That is, $\mathbf{Y}^+$ is the initial network $\mathbf{Y}(t-1)$ plus the additional ties

formed by time t , and $\mathbf{Y}^-$ is the initial network $\mathbf{Y}(t-1)$ less the ties dissolved by time t . The separable, temporal ERGM (i.e., STERGM) framework of Krivitsky and Handcock then assumes (a) a discrete-time Markov structure for the evolution of the network over time, (b) that the formation and dissolution networks are conditionally independent (i.e., separability), and (c) an exponential family form for the relevant conditional distributions. The result is a model of the form

$$\begin{aligned} \mathbb{P}_{\theta}(\mathbf{Y}(t) = \mathbf{y}(t) | \mathbf{Y}(t-1) = \mathbf{y}(t-1)) &= \mathbb{P}_{\theta}(\mathbf{Y}^+ = \mathbf{y}^+ | \mathbf{Y}(t-1) = \mathbf{y}(t-1)) \\ &\quad \times \mathbb{P}_{\theta}(\mathbf{Y}^- = \mathbf{y}^- | \mathbf{Y}(t-1) = \mathbf{y}(t-1)), \end{aligned}$$

where the two conditional distributions on the right-hand side above have exponential family form (i.e., as in equation (6.1)).

The specification of effects for these models can, like the ERGMs of Chap. 6, involve both network effects (i.e., endogenous) and effects due to vertex or edge attributes (i.e., exogenous). Maximum likelihood estimates of the corresponding parameters θ may then be calculated using extensions of the methods for standard ERGMs, based on Monte Carlo. The R package **tergm** contains a collection of functions that allow for simulation, fitting, and diagnostics with this class of models.

Alternatively, one might think to replace the discrete-time formulation in temporal ERGMs with a continuous-time formulation. The stochastic actor-oriented dynamic networks models of Snijders and colleagues do just that. See [135], for example, for a recent overview. Under these models, a continuous-time Markov chain is formulated, with each possible network corresponding to a state. The rate functions governing waiting times between state transitions (i.e., from one network to another, where the two differ in the presence or absence of a single edge) are expressed as the sum of the collection of edge-specific rates. Each of the latter are in turn represented in the manner of a generalized linear model (GLM), allowing for the incorporation of specific edge and/or vertex effects. Again, a Monte Carlo approach is required for simulation, fitting, and diagnostics. Techniques for these tasks have been developed over a period of years and are now available in the R package **RSiena**.